

Part I

Reinforcement Learning (Intro)

Author of StatQuest

1 Reinforcement Learning Via Policy Gradients

Imagine there are two choices for a snack, and these choices can vary, e.g., either size small or large. If we wanted to model this, our input would be hunger level, and model would give us the probability of the choices, but not variance in choices themselves, e.g., you might get choice A but in size small. In this setup, we do not know in advance what the output values should be to train the model. Lack of known output values can be trained using reinforcement learning, e.g., using **policy gradients**. In other words, if we have a model where we know the inputs and the probability of taking an action, but not whether that action was correctly labeled, then we cannot use standard back-propagation to train this network. However, we can use reinforcement learning to train the neural network using the **policy gradient method**.

Recall in normal back-propagation, we calculate the derivative of the residual (difference between the model's output value and the correct value) wrt the parameter to shift the value that optimizes the residual wrt the residual's magnitude. This derivative is usually calculated via gradient descent algorithm(s) like Adam. The smaller residual magnitude is better since it means the gradient descent is moving in the correct direction.

In our case, we need to provide a guess for every input because we do not have actual outputs. Assume we have a set of actions, A_i for $i = 1, 2, \dots, n$. Also, assume we have a neural network that takes a positive real number x and produces a probability using a sigmoid for the probabilities, $P(A_i|\theta, b, x)$, where b is a parameter we want to update and θ contains the other model parameters.

Example 1.1. Let A_1 and A_2 be actions. Let neural network be the structure of

$$y = \sigma(xW + b)$$

where σ is the sigmoid function,

$$\sigma(x) = \frac{e^x}{1 + e^x}$$

b is the bias, W is our weight, and $x \in [0, 1]$ is the input, and y is the probability we chose the action given the input and model parameters,

$$y = P(A_i|W, b, x)$$

If we evenly distribute our model outputs on a number line as a probability so that every output has an equal chance, we start with a seed to get an output and calculate the residual, which is the difference between the model output and the ideal output. For example, if we have two outputs A_1, A_2 , then

$$P(A_2|\theta) = 1 - P(A_1|\theta)$$

then we want the ideal outputs to be based on a degenerate probability distribution, or indicator for short, i.e.,

$$P(A|s) = \begin{cases} 1 & \text{sample puts us in A's range} \\ 0 & \text{sample put us out of A's range} \end{cases}$$

Example 1.2. Let

$$A = \begin{cases} A_1 & P(A_1|s) \in [0, 0.5] \\ A_2 & P(A_1|s) \in (0.5, 1] \end{cases}$$

then if we seed $s = 0.2$, then we chose A_1 and the ideal probability distribution would be $P(A_1|s) = 1$ and $P(A_2|s) = 0$.

We calculate the derivative of the residual (difference between ideal output and model output) wrt the variable we want to optimize. We need to update our guess based on a **reward** feedback value, e.g., -1 reward for an incorrect guess and 1 for a correct guess. This reward scales the gradient descent derivative's magnitude and also changes its direction. The sign tells you if you want to increase or decrease the probability, and the magnitude tells you by how much.

Definition 1.1. The reward, r , is the value assigned to the ideal output based on correctness, i.e., the score assigned to the action taken based on the policy.

Remark 1.1. For **policy gradient** algorithms, $r \in (0, \infty)$ for correct guesses and $r \in (-\infty, 0)$ for incorrect guesses.

Example 1.3. Let

$$r(guess) = \begin{cases} 1 & \text{guess is correct} \\ -1 & \text{guess is incorrect} \end{cases}$$

Let b be the parameter we want to optimize, Δb be the step size, A be the action we take, $0 < l$ be the learning rate, and r be the reward, then we can update the parameter via

$$\begin{aligned} b_{new} &= b_{old} - \Delta b \\ &= b_{old} - l \cdot \left(\frac{\partial P(A|s)}{\partial b_{old}} \right)_{new} \\ &= b_{old} - lr \frac{\partial P(A|s)}{b_{old}} \end{aligned}$$

When the parameter b is no longer changing wrt some tolerance, we're done, i.e., for some tolerance $0 < \varepsilon$,

$$|b_{new} - b_{old}| < \varepsilon$$

Remember, this derivative is calculated via gradient descent algorithm(s).

Observe the step size is based on the $sgn(r)$, which means the reward has to output positive values for correct guesses, and negative values for incorrect guesses, otherwise the derivative cannot change direction. In addition, if $r = 0$, then $b_{new} = b_{old}$, so r 's range in $\mathbb{R} - \{0\}$ is a mandatory requirement for policy gradients.

Example 1.4. Say you wanted ChatGPT to solve math problems. So the input is the math problem like $4x + 6 = 10$ and it spits out answers, then you just rate them based on correctness, i.e., $x = 1$.

Definition 1.2. Agent - ???

Definition 1.3. Environment - ???

Definition 1.4. policy - probabilities that come from neural network

Definition 1.5. Policy network - the neural network used to calculate the policy

The goal of reinforcement learning is to update the policy network to maximize the reward.

2 Reinforcement learning with Human Feedback

An untrained model is a model whose all parameters are just random. Assume we have known inputs and known output values.

Definition 2.1. pre-trained model - a model that is trained on a large corpus to predict tokens, e.g., an LLM.

Definition 2.2. unaligned model - a pre-trained model that is not in alignment with its intended usage, e.g., and LLM that only predicts tokens when we actually want it to give coherent responses.

Aligning a pre-trained model takes two steps, supervised fine-tuning and reinforcement learning with human feedback. Since the supervised fine-tuning dataset is hard and time consuming to construct due to the labels, it is generally much smaller than the pre-training dataset and prone to overfit due to bias. The result could be the supervised model does not generalize to new prompts that were not part of the supervised fine-tuning dataset to begin with.

Instead of spending the resources to construct a larger dataset, we can use reinforcement learning with human feedback. Our goal is to construct this large dataset by increasing the rating that the users give to the responses while minimizing cost. There are too many responses to rate, so we train a neural network called the **value neural network** to predict the rating a human will give using standard back-propagation. The architecture of this model starts off as a copy of the supervised fine-tuning model, then replace the unembedding portion that has many outputs with a single reward output.

If our input is a single prompt and our outputs are generated answers, then we can setup a feedback loop to get people to choose their preferred option based on round-robin pairs. This is our new labeled dataset. Then we feed the input prompt and labeled response to the value neural network to calculate the rewards, and the model will learn the best rewards without having to figure out how to assign them or their range in advance, the reward assignments are just parameters in our loss function.

openai used this loss function

$$-\log(\sigma(r_{better} - r_{worse}))$$

to give the reward, where σ is the sigmoid function, i.e.,

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Recall sigmoid returns 1 for large positive inputs. Also, recall

$$\lim_{x \rightarrow 1} \log(x) = 0$$

This means we want the worse reward to end up being very small, so effectively we want the value reward network to return the best responses. We also have a negative because we want the highest reward, but gradient descent optimizes for the lowest loss value in magnitude, so we have to flip the sign of the output.

Finally, we can use this trained value reward network to update our supervised fine-tuned model in the following way. Pass an input to the supervised fine-tuned model, generate responses, and pass each (input, output) pair to the reward model to calculate the rewards. And then use these rewards to update the parameters using the policy gradient algorithm. The resultant model is considered **aligned** to our use case.

3 References

- <https://arxiv.org/abs/1707.06347>
- <https://arxiv.org/abs/1706.03741>
- <http://incompleteideas.net/book/the-book-2nd.html> – canonical reinforcement learning book
- <https://arxiv.org/pdf/2203.02155> – OpenAI’s paper on training language models with human feedback